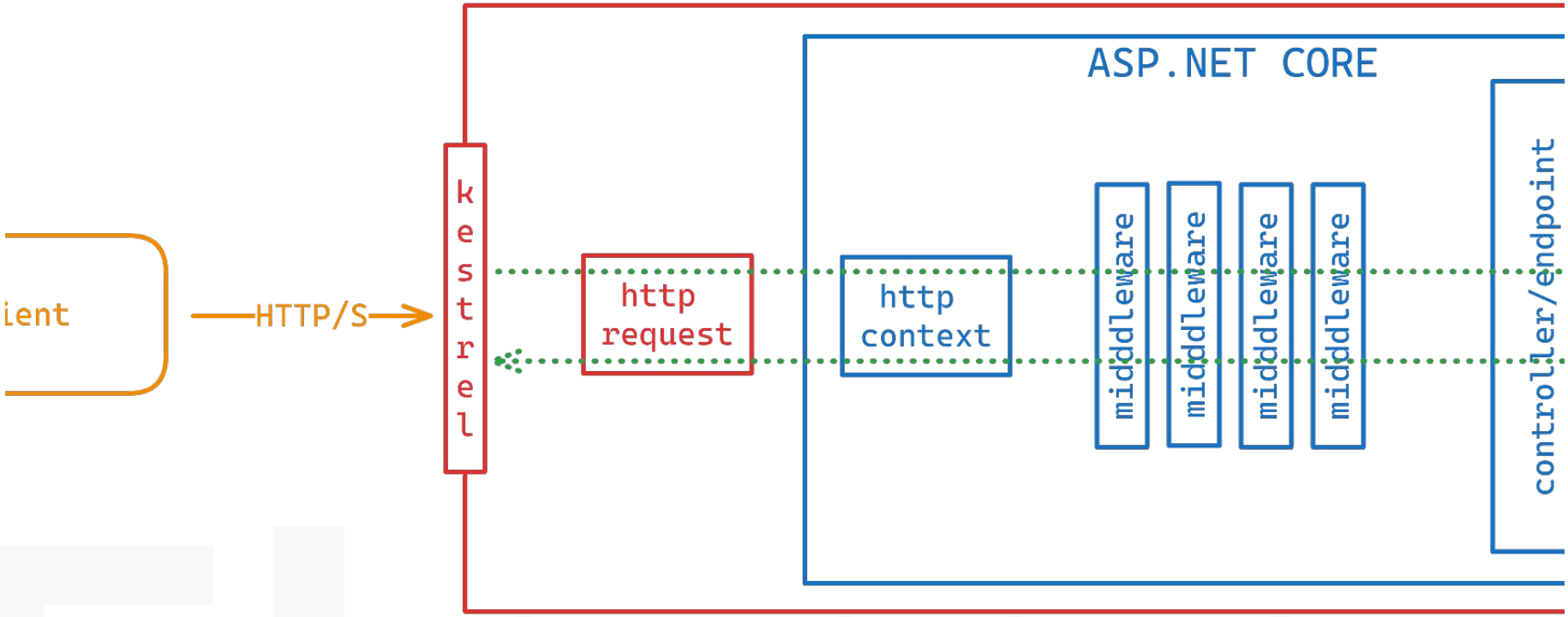
C# DO JEITO CERTO
MÓDULO 01

AULA 03

ASPNET CORE O QUE MUDA O JOGO

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

KESTREL



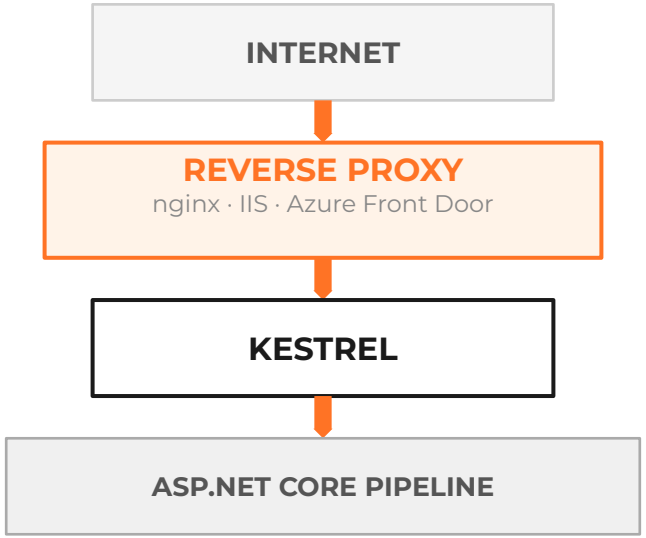
Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

KESTREL E O REVERSE PROXY

Como o ASP.NET Core recebe requisições em produção

- Servidor HTTP embutido no .NET — multiplataforma
- Não possui reverse proxy nativo **(ao contrário do IIS)**
- Em produção: sempre um proxy reverso na frente

> **Expor o Kestrel diretamente em produção é um risco de segurança e de operação.**



Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

CONTROLLER-BASED vs MINIMAL API

Escolha baseada em contexto, não em hype

CONTROLLER-BASED

- Times grandes
- Rotas complexas
- Filters
- Versioning
- Middleware rico
- Open API nativo

Estrutura que escala com o time

VS

MINIMAL API

- Microsserviços
- Endpoints simples
- Prototipagem
- Alta performance
- Poucas dependências
- Delegates / Lambda

Velocidade que não escala com complexidade

Minimal API em projeto complexo = dívida técnica disfarçada de modernidade

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

WEBAPPLICATIONBUILDER

A separação fundamental: Services ≠ Pipeline

MOMENTO 1 — SERVICES

```
var builder =
  WebApplication.CreateBuilder(args);

// Registra serviços no container
builder.Services.AddControllers();
builder.Services
  .AddApplicationServices(
      builder.Configuration);
builder.Services
  .AddPersistence(
      builder.Configuration);
```

MOMENTO 2 — PIPELINE

```
// Pipeline HTTP — ordem importa!
// Cada Use() envolve o próximo

app.UseExceptionHandler(); // ①
app.UseAuthentication();   // ②
app.UseAuthorization();    // ③

app.MapControllers();

app.Run();
```

```
var app = builder.Build();
```

← *container fechado, pipeline abre*

Registra dependências · Imutável após Build()

Ordem importa · Middleware executado em sequência

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

DI LIFETIMES

Observe o comportamento de cada instância por lifetime

Lifetime	Request A	Request B	Request C	*Insight*
Singleton	inst. 1 — mesma instância para todas as requests			**Vive enquanto a app rodar** *Atenção com estado mutável*
Scoped	inst. 2	inst. 3	inst. 4	**1 por request HTTP** *Descartada ao final. Ideal para DbContext.*
Transient	inst. 5	inst. 8	inst. 11	**Nova a cada injeção** *3 requests × 3 pontos =* **9 simultâneas**
	inst. 6	inst. 9	inst. 12	
	inst. 7	inst. 10	inst. 13	

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

CAPTIVE DEPENDENCY

O bug que aparece em produção — Singleton capturando Scoped

Captive Dependency

```
// ReportService é Singleton mas injeta Scoped
public class ReportService(AppDbContext db)
    : IReportService { }

builder.Services
    .AddSingleton<IReportService, ReportService>();

builder.Services
    .AddDbContext<AppDbContext>(...);
    // ↑ Scoped por padrão
```

O QUE ACONTECE

ReportService
Singleton — vive para sempre

captura

AppDbContext
Scoped — deveria morrer por request

⚠️ **DbContext nunca descartado**

✅ **Dev: funciona**
Poucas requests, sem concorrência. Bug não se manifesta.

❌ **Prod sob carga**
DbContext entre threads → race condition.

💣 **Exception**
`InvalidOperationException`
"A second operation was started"

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

A REGRA DE DEPENDÊNCIA

Uma dependência nunca pode ter lifetime mais curto que quem a consome

PODE depender de

- **Singleton** → **Singleton**
- **Scoped** → **Singleton**, **Scoped**
- **Transient** → **Singleton**, **Scoped**, **Transient**

NÃO PODE depender de

- **Singleton** → **Scoped**
  captive dependency — DbContext nunca descartado
- **Singleton** → **Transient**
  Transient vira Singleton na prática

REGRA PARA MEMORIZAR

"Uma dependência nunca pode ter lifetime mais curto que quem a consome."

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

DETECTANDO CAPTIVE DEPENDENCIES

A validação que o framework já oferece — e o risco de não habilitá-la em produção

UseDefaultServiceProvider — configurar validação de scopes

```
// ASP.NET Core valida em Development por padrão
// Em Production, a validação é DESABILITADA ← o perigo

// Para habilitar em todos os ambientes:
builder.Host.UseDefaultServiceProvider(options =>
{
    options.ValidateScopes = true;
    options.ValidateOnBuild = true;
    // falha na startup, não em runtime
});
```

DEVELOPMENT

ValidateScopes: ON
Captive dependency detectada na 1ª request.

PRODUCTION — o perigo

ValidateScopes: OFF
Bug silencioso até a 1ª request concorrente.

ValidateOnBuild

Falha na startup
Erro antes de qualquer request. Habilite sempre.

RECOMENDAÇÃO

Habilite ValidateScopes e ValidateOnBuild em todos os ambientes — erros de DI devem explodir na startup, não em produção.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

ACTION RESULTS E PROBLEMDETAILS

Padronizar respostas de erro com o contrato RFC 7807

Action Results — retornos mais usados

```
// Sucesso
return Ok(dto);                    // 200
return Created(uri, dto);            // 201
return CreatedAtAction(...);         // 201 + Location
return NoContent();                 // 204

// Erro
return NotFound();                  // 404
return Conflict();                  // 409
return Problem("msg", statusCode: 422); // ProblemDetails
```

ProblemDetails — RFC 7807

```
// Contrato de resposta (RFC 7807):
{
  "type": "https://.../rfc7807",
  "title": "Validation failed",
  "status": 422,
  "instance": "/api/v1/accounts",
  "detail": "Email já cadastrado.",
  "errors": {
    "email": ["E-mail inválido"]
  }
}
```

PADRÃO

Use sempre Problem() para erros — não retorne strings soltas.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

GLOBAL EXCEPTION HANDLER

Eliminar try/catch nos controllers com um único ponto de tratamento

GlobalExceptionHandler : IExceptionHandler

```
public class GlobalExceptionHandler : IExceptionHandler
{
  public async ValueTask<bool> TryHandleAsync
    (HttpContext ctx, Exception ex, CancellationToken ct)
  {
    var (status, title) = ex switch
    {
      NotFoundException   => (404, "Resource not found"),
      ValidationException => (422, ex.Message),
      _                   => (500, "An unexpected error")
    };

    ctx.Response.StatusCode = status;
    await ctx.Response.WriteAsJsonAsync
      (new ProblemDetails { Title=title, Status=status }, ct);

    return true; // exception tratada, não propaga
  }
}
```

Registro

```
builder.Services
  .AddExceptionHandler<GlobalExceptionHandler>();
builder.Services
  .AddProblemDetails();

app.UseExceptionHandler();
// ANTES de MapControllers
```

Erro clássico

```
// Ordem errada:
app.MapControllers();
app.UseExceptionHandler();

// Resultado:
// UseExceptionHandler não
// pega nada neste caso
```

ORDEM IMPORTA

UseExceptionHandler() ANTES de MapControllers() — middlewares são executados na ordem de registro.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

BACKGROUND SERVICES: QUANDO E POR QUÊ

O problema que BackgroundService resolve — antes de qualquer código

Comunicação

1
E-mail transacional
confirmação, senha, alertas

2
Notificações push
mobile, web push, webhooks

Dados

1
Consumo de fila
RabbitMQ, Azure SB, SQS

2
Sincronização externa
ERP, CRM, APIs de parceiros

Operações

1
Relatórios agendados
geração periódica, exportação

2
Limpeza e warm-up
dados expirados, cache startup

PRINCÍPIO
Não faça o usuário esperar o que pode ser feito em background. Retorne rápido, processe depois.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

IHostedService vs BackgroundService

Quando usar cada um — sem deixar dúvida

IHostedService — controle total

```
// Você implementa startup e shutdown
public class MyService : IHostedService
{
  public async Task StartAsync
    (CancellationToken ct)
  {
    await InitializeAsync(ct);
  }
  public async Task StopAsync
    (CancellationToken ct)
  {
    await CleanupAsync(ct);
  }
}
```

BackgroundService — 90% dos casos

```
// Herda IHostedService — só o loop principal
public class MyService : BackgroundService
{
  protected override async Task
    ExecuteAsync(CancellationToken ct)
  {
    while (!ct.IsCancellationRequested)
    {
      await DoWorkAsync(ct);
      await Task.Delay(30_000, ct);
    }
  }
}
```

IHostedService — usar quando:
startup/shutdown customizado ou liberação de recursos em ordem

BackgroundService — usar para:
loops periódicos, consumo de fila, processamento contínuo em background

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

THE SCOPE PROBLEM

BackgroundService (Singleton) não pode depender de DbContext (Scoped) — o mesmo captive dependency do Bloco 1

BackgroundService → Singleton capturando Scoped

```
// BackgroundService é registrado como SINGLETON pelo host
// DbContext é SCOPED
// ← É exatamente o captive dependency que vimos antes

// ❌ Isso vai lançar exceção em runtime:
public class ReportService
    (AppDbContext db) : BackgroundService
{
  protected override async Task ExecuteAsync
      (CancellationToken ct)
  {
    // db preso no singleton — corrompe dados em concorrência
  }
}
```

O que acontece

Host inicia ReportService
registrado como Singleton

↓

Tenta resolver AppDbContext
que é Scoped

↓

❌ **InvalidOperationException**
Cannot consume scoped service

🔗 **Conexão com o Bloco 1**
Mesmo problema que vimos na regra de dependência — Singleton não pode depender de Scoped. A solução: IServiceScopeFactory.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

IServiceScopeFactory — a solução

Criando escopos manualmente dentro do BackgroundService

Padrão correto — IServiceScopeFactory

```
public class ReportService
    (IServiceScopeFactory scopeFactory)
    : BackgroundService
{
  protected override async Task ExecuteAsync
      (CancellationToken ct)
  {
    while (!ct.IsCancellationRequested)
    {
      await ProcessAsync(ct);
      await Task.Delay(TimeSpan.FromMinutes(5), ct);
    }
  }

  private async Task ProcessAsync(CancellationToken ct)
  {
    // Criar scope — descartar ao final
    await using var scope = scopeFactory.CreateAsyncScope();

    var repo = scope.ServiceProvider.GetRequiredService<IReportRepository>();
    var db   = scope.ServiceProvider.GetRequiredService<AppDbContext>();

    var pending = await repo.GetPendingAsync(ct);
    await db.SaveChangesAsync(ct);
  } // scope descartado → DbContext descartado junto
}
```

IServiceScopeFactory
`Singleton seguro — criado uma vez`
injetado no construtor normalmente

await using
`CreateAsyncScope()`
garante DisposeAsync correto. Nunca só using — perde o async dispose

scope.ServiceProvider
`GetRequiredService<T>()`
resolve Scoped dentro do escopo — ciclo de vida correto

} // fim do ProcessAsync
scope → DbContext descartado autom.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Multi-Tenancy: o Problema

O anti-padrão que o middleware resolve

Anti-padrão — tenant resolvido em cada controller

```
// ❌ Duplicado em todo controller
public class AccountsController : ControllerBase
{
  [HttpGet]
  public async Task<IActionResult> GetAll()
  {
    var tenantId = Request.Headers["X-Tenant-Id"
                  .FirstOrDefault();
    if (tenantId is null) return BadRequest();

    var tenant = await _tenantStore.
        GetByIdAsync(tenantId);

    // ... lógica duplicada em cada controller
  }
}
```

Duplicação
Mesma lógica em todo controller

Fácil de esquecer
Um controller sem a verificação expõe dados errados

Difícil de testar
Dependência do Request em cada método

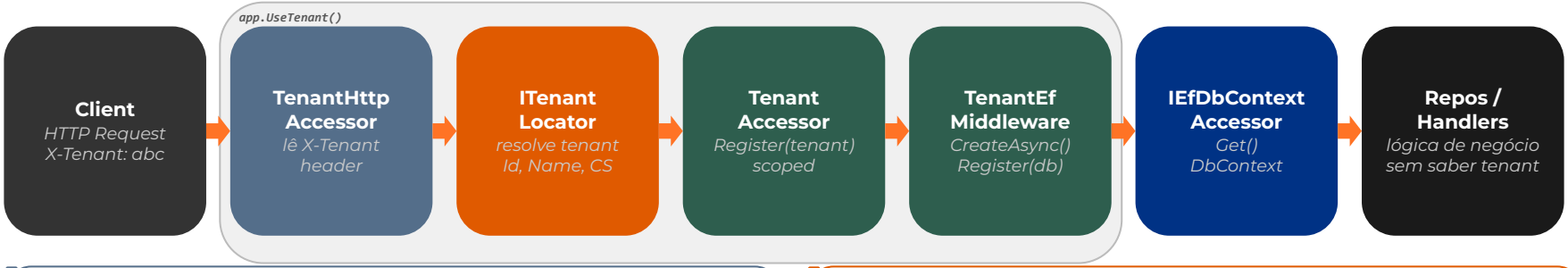
Connection string
Onde fica a do tenant? No quê?

A solução: um middleware único resolve o tenant e injeta no contexto
Todos os controllers recebem via DI — sem saber como foi resolvido, sem duplicação

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Resolução de Tenant — Pipeline

Como o tenant é resolvido por request



1 Ler tenant do request

DefaultTenantHttpAccessor lê X-Tenant.
Fallback: "default" se ausente.

2 Resolver (ITenantLocator)

ConfigTenantLocator → GymErpTenant
{ Id, Name, ConnectionString }

3 Registrar DbContext no escopo

factory.CriarAsync(name) → accessor.Register(db)
Clear() ao fim do request.

4 Uso via IEfDbContextAccessor

accessor.Get() nos repos e handlers.
Nunca recebem DbContext direto.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Sequência de Resolução

Do HTTP request ao DbContext nos repositórios — passo a passo

TenantHttpAccessor — 1

Lê header X-Tenant do request

DefaultTenantHttpAccessor.Get() → tenantKey = "default" (ou valor do header)

→ *tenantKey: string*

ITenantLocator — 2

Resolve Tenant

tenantLocator.Get(tenantKey, ct) → ConfigTenantLocator lê do appsettings

→ *Tenant { Id, Name, ConnectionString }*

TenantAccessor — 3

Registra o tenant no escopo do request

TenantAccessorRegisterMiddleware chama tenantAccessor.Register(tenant)

→ *tenant disponível via ITenantAccessor no escopo*

TenantEfMiddleware — 4

Cria e registra o DbContext do tenant

factory.CriarAsync(tenant.Name) → accessor.Register(dbContext)

→ *IEfDbContextAccessor<ConcreteDbContext> pronto*

Dispose — 5

Descarta ao fim do request

accessor.Clear() faz dispose do DbContext · TenantAccessor.Dispose() limpa o tenant

→ *sem leak de conexão ou contexto*

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com